

Apex Institute Learning Management Platform

Team Development & Simultaneous Collaboration Standard Operating Procedure (SOP)

Component	Live URL / Endpoint	Deployment Host & Environment
Frontend Web Application	https://sathish9042.github.io/ONLINE-COURSE/	GitHub Pages (Auto CI/CD on push to main)
Backend REST API	https://online-course-47df.onrender.com	Render Free Web Service (Auto deploy from main)
PostgreSQL Database	Neon.tech Cloud Serverless PostgreSQL	AWS Ohio (us-east-2) with SSL encryption
Git Code Repository	https://github.com/SATHISH9042/ONLINE-COURSE	Branches: main (Production), gh-pages (Web)

Default Pre-Seeded Test Credentials

Role	Login Identifier	Password	Clearance & Portal Permissions
Administrator	admin@institute.edu +919999999999	Admin@123	ACTIVE — Full Executive Console, Clearances, Curriculum
Active Student	priya.patel@example.com +919888877777	Student@123	ACTIVE — Video Player, Coding Sandbox, MCQ Assessments
Pending Student	+919876543210	Student@123	PENDING_APPROVAL — Restricted clearance lockout flow

1. Initial Environment Setup (One-Time for Each Team Member)

Each team member must set up their local workstation using the following commands:

```
# 1. Clone repository from GitHub
git clone https://github.com/SATHISH9042/ONLINE-COURSE.git
cd ONLINE-COURSE

# 2. Install backend and frontend dependencies
cd backend && npm install
cd ../frontend && npm install
cd ..

# 3. Initialize local embedded zero-config PostgreSQL database
npm --prefix backend run migrate
npm --prefix backend run seed
```

2. Daily Development Workflow (Single Developer)

Always update your local code and create a dedicated branch before making modifications:

```
# Fetch latest main branch
git checkout main
git pull origin main

# Create a feature branch for your specific task
git checkout -b feature/your-task-name
```

Run the development servers locally in two terminal windows:

```
# Terminal 1: Backend API Server (http://localhost:5001)
npm --prefix backend run dev

# Terminal 2: Frontend React Application (http://localhost:5173)
npm --prefix frontend run dev
```

Where to make modifications:

- **Frontend UI & Pages:** Located in `frontend/src/pages/` and `frontend/src/components/`. Changes reload instantly in the browser.
- **Backend Logic & APIs:** Located in `backend/src/routes/`, `backend/src/controllers/`, and `backend/src/validators/`.
- **Database Schema:** Update `backend/src/database/schema.sql`, then run `npm --prefix backend run migrate` to verify.

Verification before pushing:

Always ensure all 118 automated tests pass and the frontend compiles cleanly:

```
# 1. Run all 10 phases of automated tests (118 assertions)
npm --prefix backend test

# 2. Verify TypeScript frontend build
npm --prefix frontend run build
```

3. Simultaneous Multi-Developer Collaboration (Person A & Person B)

When two or more developers work on the repository at the same time, follow this strict procedure to guarantee zero code loss and zero merge conflicts.

The 3 Golden Rules of Parallel Work

Rule 1: NEVER commit directly to 'main'. Main is connected to live production.

Rule 2: Every feature gets its own branch (e.g., *feature/person-a-task* and *feature/person-b-task*).

Rule 3: Separate files & concerns. Assign distinct features to avoid simultaneous edits to the same lines.

Chronological Walkthrough: Person A and Person B in Action

Step	Person A Action (e.g., Certificate Feature)	Person B Action (e.g., Profile Photo Upload)
Step 1: Morning Sync	git checkout main git pull origin main git checkout -b feature/certificate	git checkout main git pull origin main git checkout -b feature/avatar-upload
Step 2: Local Work	Works in frontend/src/pages/learning/ Runs local servers; tests feature.	Works in frontend/src/pages/student/ Runs local servers; tests feature.
Step 3: Person A Finishes First	git add . git commit -m "feat: certificate" git push -u origin feature/certificate Opens PR on GitHub & Merges into main. <i>Render & GitHub Pages deploy automatically!</i>	Continues developing on their local branch. (Person B's local code is completely unaffected by Person A's push).
Step 4: Person B Syncs Latest main (CRITICAL)	Starts next task or reviews teammate's PR.	Before finishing, Person B pulls Person A's new changes into their own working branch: git fetch origin git merge origin/main <i>Person B now has both their code AND Person A's code!</i>
Step 5: Person B Pushes & Merges	Reviews Person B's Pull Request.	npm --prefix backend test git add . git commit -m "feat: avatar upload" git push origin feature/avatar-upload Opens PR on GitHub & Merges into main.

4. How to Resolve Merge Conflicts (When Both Edit the Same File)

If Person A and Person B modify the **exact same lines** in the same file, Git pauses during `git merge origin/main` and alerts you:

```
<<<<<< HEAD (Current Change: Person B's code)
const apiBase = "https://online-course-47df.onrender.com/api/v1";
=====
const apiBase = process.env.VITE_API_URL || "https://online-course-47df.onrender.com/api/v1";
>>>>>> origin/main (Incoming Change: Person A's code)
```

Resolution Steps in Visual Studio Code:

1. Open the highlighted file in VS Code.

2. Above the conflict markers, click one of the interactive options:
 - **Accept Current Change:** Keeps Person B's code.
 - **Accept Incoming Change:** Keeps Person A's code from main.
 - **Accept Both Changes:** Preserves both modifications sequentially.
3. Save the file and complete the merge commit:

```
git add .
git commit -m "merge: resolve conflicts with main"
git push origin feature/your-branch-name
```

5. Automated CI/CD Deployment Architecture

When code is merged into `main`, two automated pipelines execute simultaneously:

Deployment Target	Trigger Condition	Automated Actions Performed
Backend Web Service Render.com	Any commit pushed to <code>main</code> touching backend/	1. Installs Node.js dependencies 2. Compiles TypeScript into <code>dist/</code> 3. Connects to Neon PostgreSQL 4. Runs schema migrations and seeds 5. Launches API on HTTPS port 443
Frontend Web Application GitHub Pages	Any commit pushed to <code>main</code>	1. GitHub Actions workflow triggers (<code>deploy.yml</code>) 2. Injects production backend URL 3. Compiles production bundle with Vite 4. Pushes static bundle to <code>gh-pages</code> branch 5. Live site reflects changes in 60 seconds

6. Team Task Tracking (GitHub Projects)

To ensure team members never accidentally work on the same component at the same time:

- Open the Kanban board at: <https://github.com/SATHISH9042/ONLINE-COURSE/projects>
- Maintain three standard columns: **To Do**, **In Progress**, and **Done**.
- Assign tasks to specific developers before writing any code.